

PDF-4: Engine Core (OPS-1 Diagnostic Kernel)

1 Abstract

This document defines the *Engine Core* as a **diagnostic-only kernel** within the OPS-1 layer. The Engine Core is specified as a minimal, non-adaptive, non-intervening evaluator that assesses immutable structural state snapshots against explicitly declared invariants and terminates evaluation irreversibly when STOP conditions apply.

PDF-4 strictly restricts the Engine Core to diagnostic semantics. The kernel introduces no control logic, no optimization objectives, no learning or adaptation mechanisms, and no prescriptive, managerial, or interpretive layer. All admissible operations are closed, explicitly declared, and structurally bounded.

The Engine Core is defined as a formally constrained mapping from declared inputs to declared diagnostic outputs. Its sole function is to determine structural admissibility or non-admissibility with respect to invariants and thresholds. STOP is treated as a first-class terminal boundary, not as an error, warning, or recoverable condition.

This artefact provides a normative diagnostic anchor for higher-level tooling and documents. Directionality is strictly one-way: OPS-1 definitions may be referenced by external artefacts, while PDF-4 itself does not depend on or inherit operational, methodological, or governance semantics.

PDF-4 does not specify implementations, execution environments, workflows, or organizational processes. It defines the Engine Core exclusively as a self-contained diagnostic boundary.

2 Problem Statement

2.1 Why an Engine Core artefact exists

In large-scale systems and organizations, diagnostic mechanisms are frequently entangled with control, optimization, or governance processes. As a consequence, diagnostic artefacts inherit implicit objectives, adaptive behavior, and prescriptive semantics that obscure their epistemic status.

The Engine Core artefact exists to enforce a strict separation between *diagnosis* and *intervention*. It defines a formally constrained diagnostic kernel whose outputs are limited to structural classification and irreversible STOP boundaries. By construction, the Engine Core cannot recommend actions, optimize outcomes, or modify its own behavior.

This separation is not a design preference but a structural requirement. Without an explicitly bounded diagnostic kernel, diagnostic results become inseparable from the control or governance mechanisms that consume them, and their validity cannot be assessed independently.

The Engine Core therefore functions as a stable diagnostic anchor. Higher-level tooling may consume its outputs, but responsibility for interpretation, decision-making, and action attribution remains strictly external to the kernel.

2.2 What is explicitly out of scope

PDF-4 intentionally excludes all concerns that would transform diagnosis into control, prescription, or interpretation.

Out-of-scope domains. The Engine Core does not address:

- decision-making, prioritization, or trade-off resolution;
- transformation planning, roadmaps, or execution sequencing;
- organizational roles, accountability, or governance structures;
- performance management, KPIs, or success criteria;
- user interaction, facilitation, or interpretive guidance.

These concerns may be handled by external artefacts or processes, but they must not be embedded into the Engine Core itself.

Deliberate incompleteness. The Engine Core does not aim to be a complete system or organizational model. Its incompleteness is structural, not incidental: the artefact derives its diagnostic validity from the explicit refusal to encode control, optimization, or prescriptive semantics.

3 Scope and Non-Claims

3.1 Diagnostic-only boundary

This artefact defines the *Engine Core* as a **diagnostic-only kernel**. Its scope is fixed, closed, and exhaustively limited to: (i) accepting a declared structural state snapshot, (ii) evaluating that snapshot against a declared invariant set, (iii) emitting a structural classification, and (iv) irreversibly terminating evaluation when terminal STOP conditions apply.

The Engine Core has **no causal influence** on the diagnosed system (see `I_nonintervention`). It is not an actuator, controller, optimizer, governance mechanism, decision authority, or intervention layer. Any embedding or interpretation as such is invalid (see `P_control`, `P_prescription`).

Closed operation set (structural invariant). The following operations exhaust the admissible behavior of the kernel:

- **OBSERVE:** accept a read-only state snapshot;
- **EVALUATE:** evaluate declared invariants on that snapshot;
- **CLASSIFY:** map invariant satisfaction to a finite structural label;
- **STOP/HALT:** irreversibly terminate evaluation for the given case.

These operations denote structural roles, not procedural steps. No additional operations, phases, execution controls, or behavioral constructs exist within the kernel (see `I_nonexp`, `P_workflow`).

3.2 OPS-1 contract vs. toolkits (OPS-2)

OPS-1 is an architecture-and-contract layer. It defines admissibility, invariants, prohibitions, determinism, reproducibility, and provenance identity. OPS-1 does not define usage, tooling, workflows, interfaces, or decisions (see `I_diagonly`).

Any toolkit surface (CLI/GUI/scripts/integrations) belongs to OPS-2 and is, by definition, an exchangeable consumer of the OPS-1 contract. OPS-2 may change independently; OPS-1 must not be defined, reinterpreted, or operationalized by OPS-2. Any dependency in the opposite direction is invalid.

Binary compliance. OPS-1 compliance is binary: conformant or non-ESTRA (see `I_binary`, `D_graded`). Intent, usability goals, or product considerations are irrelevant.

Non-normativity of toolkits. Toolkits are never normative. They must not introduce new semantic degrees of freedom, reinterpret terminal STOP, weaken invariants, or transform structural labels into prescriptions (see `P_policy`, `P_prescription`).

Auto-disqualification. Any representation of OPS-1 as a “method”, “tool”, “framework”, “best practice”, “playbook”, or “how-to” is a category error. Any consumer layer that induces or implies “you should do X” from OPS-1 output or from terminal STOP is non-conformant and therefore non-ESTRA (see `D_adaptive_ext`).

3.3 No-prescription / no-management claims

This document contains no prescriptive and no managerial statements by design.

Explicit non-claims. The Engine Core does not:

- recommend actions, remediations, or interventions;
- define roadmaps, sequences, or execution plans;
- set targets, KPIs, objectives, or success criteria;
- prioritize options, rank alternatives, or compare desirability;
- allocate resources or define governance or control processes;
- provide best practices, frameworks, methodologies, or transformation guidance.

Any process or decision-making that consumes diagnostic output is external to the Engine Core and outside OPS-1 authority (see `P_policy`, `P_prescription`).

3.4 External Dependencies and Assumptions

OPS-1 is intentionally minimal. It assumes the existence of external artefacts but does not inherit their semantics implicitly.

Mandatory dependencies. The following artefacts are required to instantiate OPS-1 semantics:

- **ETS.K_EA**: provides the semantic symbol space for facts, invariants, and the STOP token.
- **OPS-0**: defines the non-prescriptive, diagnostic-only boundary.

Absence of a mandatory dependency means OPS-1 cannot be instantiated, but does not invalidate the specification itself.

Contextual dependencies (non-normative). The following artefacts provide background context only:

- **OC Core**: ontological grounding;
- **JPHQ (monograph)**: extended formal context.

OPS-1 does not inherit semantics from contextual artefacts automatically.

Explicitly not required. OPS-1 does not require OPS-2 or higher layers, any EA framework, any implementation, dataset, benchmark, or runtime environment.

Traceability note. This document may declare a SHA-256 hash for provenance. Verification requires access to the originating artefact. If unavailable, the hash functions as a provenance identifier, not as a reproducibility guarantee.

3.5 What the engine does not infer

The Engine Core does not infer missing structure or enrich semantics. It operates exclusively on explicitly declared inputs.

Forbidden inference classes. The kernel does not:

- infer intent, priorities, accountability, or organizational motives;
- infer causal narratives beyond declared invariant reference;
- forecast future states or simulate counterfactual scenarios;
- learn from history or adapt thresholds, invariants, or rules;
- complete missing observability via assumptions, heuristics, or estimation.

Structural determinacy (descriptive). For identical declared inputs $(S, \mathcal{I}, \Theta)$, the kernel yields the same classification and terminal STOP decision (see **I_det**). This claim applies only to kernel behavior and makes no claim about determinism of the world, organizations, or outcomes.

STOP as boundary, not signal. Terminal STOP is not advice, not an error, and not a recoverable warning. It denotes a structural boundary beyond which further diagnostic evaluation is formally non-admissible for the given case (see **I_stop_mono**).

4 Core Objects and Invariants

This section defines the *minimal* object model of the Engine Core as a **diagnostic-only** kernel. All objects introduced here are strictly **structural**: they encode admissibility and terminal boundaries only. They carry no managerial, prescriptive, predictive, optimization, or decision semantics.

4.1 Normative Core (authoritative)

This subsection is the **sole authoritative source** of OPS-1 norms for the Engine Core. All other sections **must reference** these identifiers (I_* , P_* , D_*) and **must not** restate norms except by explicit reference.

4.1.1 Kernel invariants (I_*)

Kernel invariants are necessary conditions for the Engine Core to exist as an OPS-1 diagnostic kernel. Violation of any kernel invariant implies terminal invalidity (non-OPS-1). Kernel invariants are **not** the same as declared invariants $\mathcal{I}$ (which are external predicates over S ; see below).

STOP trigger (structural). Terminal STOP is determined by a *declared* STOP predicate over the declared inputs and the evaluation outcome:

$$\text{STOP} := \text{STOP_predicate}(S, \mathcal{I}, \Theta, \mathcal{O}, \mathbf{v}) \in \{0, 1\}.$$

Kernel invariants constrain admissibility of the evaluation itself; if a kernel invariant is violated, the evaluation is non-admissible and terminal STOP applies.

$I_{\text{nonintervention}}$ Non-intervention: the kernel has no causal influence on the diagnosed system.

$I_{\text{nonadaptive}}$ Non-adaptivity: kernel behavior does not change as a function of past evaluations.

I_{det} Determinism: for identical $(S, \mathcal{I}, \Theta)$ the kernel yields the same R .

$I_{\text{inv_transparent}}$ Invariant transparency: all applied invariants are explicitly declared and inspectable.

I_{prov} Provenance preservation: each result is traceable to $(S, \mathcal{I}, \Theta)$ via an append-only record.

I_{nonexp} Non-expansiveness: the kernel introduces no new primitives, axes, or semantics during operation.

I_{diagonly} Diagnostic-only outputs: outputs express admissibility/invalidity only and contain no prescriptions.

$I_{\text{registry_nodecision}}$ Registry non-decisionality: the registry contains references only and must not participate in evaluation, classification, or STOP semantics.

$I_{\text{evidence_immutable}}$ Evidence immutability: referenced evidence artefacts are immutable (no overwrite/replace/delete).

I_stop_mono Monotonic terminal STOP: once STOP holds for a case, no further diagnostic state exists for that case.

I_binary Binary conformance only: conformance is defined as valid/invalid, with no graded or partial status.

4.1.2 Prohibitions (P_*)

Prohibitions define forbidden semantic classes. If any prohibited semantic is introduced (explicitly or implicitly), the artefact is non-OPS-1.

P_feedback Any feedback loop semantics.

P_learning Any learning, adaptation, or stateful tuning.

P_optimization Any optimization objective or target-seeking.

P_prescription Any recommendation, prescription, or intervention.

P_control Any control-action semantics (actuation).

P_workflow Any workflow/UX/process semantics inside OPS-1.

P_policy Any policy or decision semantics (including prioritization).

4.1.3 Auto-disqualifying conditions (D_*)

Auto-disqualifying conditions are binary triggers that imply immediate STOP and invalidate OPS-1 conformance. They are not “warnings” and admit no recovery.

D_nondet Non-deterministic execution violates **I_det**.

D_prov_loss Provenance loss violates **I_prov**.

D_evidence_mut Evidence mutation violates **I_evidence_immutable**.

D_registry_decision Registry participates in semantics, violating **I_registry_nodecision**.

D_adaptive_ext Any extension introduces **P_feedback** or **P_learning**, violating **I_nonadaptive**.

D_graded Any graded/partial conformance claim violates **I_binary**.

4.1.4 Minimal proof sketches (non-exhaustive)

These sketches are provided to justify the necessity of the above constraints. They introduce no additional semantics.

STOP monotonicity. OPS-1 validity is defined by a finite set of binary constraints and a terminal STOP predicate. Since OPS-1 provides no operator for restoring violated kernel constraints and admits no transitions out of STOP, once STOP holds, STOP is terminal by construction.

Cycles imply feedback. A cycle in an operation graph implies that some output of an earlier step can influence a later invocation of the same step. This introduces dependency on past execution states, which is feedback semantics (`P_feedback`) and therefore violates non-adaptivity (`I_nonadaptive`).

Binary conformance is necessary. Any graded conformance requires weighting, prioritization, or trade-offs among violations. Such mechanisms introduce decision/optimization semantics (`P_policy`, `P_optimization`), hence OPS-1 restricts conformance to a binary valid/invalid judgment (`I_binary`).

4.2 Data model (entities)

State snapshot. A *state snapshot* is an immutable input object:

$$S := \langle \mathcal{F}, \mathcal{V}, \mathcal{O} \rangle$$

where:

- $\mathcal{F}$ is a finite set of observable fields (named, typed);
- $\mathcal{V}$ is a valuation mapping fields to values (possibly partial);
- $\mathcal{O}$ is an observability descriptor indicating which values are known, unknown, or indeterminate, and why.

The Engine Core treats S as read-only and does not infer missing values.

Invariant set. An *invariant* is a declared predicate over the snapshot:

$$I_k : S \rightarrow \{0, 1\}$$

The *invariant set* is a finite, explicitly declared collection:

$$\mathcal{I} := \{I_1, I_2, \dots, I_n\}$$

Invariants are external declarations consumed by the kernel. The kernel does not synthesize, refine, adapt, or reinterpret invariants (see `I_inv_transparent`, `I_nonadaptive`).

Threshold objects. Thresholds are declared boundary parameters referenced by invariants:

$$\Theta := \{\theta_1, \theta_2, \dots\}$$

A threshold object is a typed and versioned parameter, scalar or structured. Thresholds are evaluation inputs and are not tuned, optimized, calibrated, or learned (see `P_optimization`, `P_learning`).

Evaluation result. Given $(S, \mathcal{I}, \Theta)$, evaluation yields a structural result:

$$R := \langle \mathbf{v}, \mathbf{c}, \text{STOP} \rangle$$

where:

- $\mathbf{v} \in \{0,1\}^n$ is the vector of invariant satisfaction bits;
- $\mathbf{c}$ is a finite structural classification label;
- $\text{STOP} \in \{0,1\}$ is the terminal halt flag.

The result encodes diagnostic admissibility only (see `I_diagonly`, `P_prescription`, `P_policy`).

Provenance record (minimal). For each evaluation, the Engine Core produces an **append-only** provenance record:

$$P := \langle h(S), h(\mathcal{I}), h(\Theta), \mathbf{v}, \mathbf{c}, \text{STOP} \rangle$$

where $h(\cdot)$ denotes stable identifiers (hashes or canonical digests). The record supports traceability and conformance checking only (see `I_prov`).

Registry (append-only registry; logical reference space). A *registry* is a logical reference space whose sole role is to *declare and preserve* references to artefacts related to an evaluation. The registry carries **references only** (see `I_registry_nodecision`).

Registry entries (types, without formats). A registry entry is an append-only reference record. The Engine Core requires only that entries be uniquely identifiable and stably referencable. Entry types include:

- **Identity entries:** stable identifiers for S , $\mathcal{I}$, Θ , and the Engine Core version.
- **Artefact reference entries:** references to evidence artefacts and auxiliary observational artefacts (e.g. run logs), without semantic use.
- **Provenance link entries:** references establishing a complete mapping from each diagnostic output to its declared inputs and referenced evidence artefacts.
- **Conformance entries:** references asserting that kernel invariants and strict warning gates were satisfied for a produced output.

Registry entries are append-only. Once a reference exists, it must not be mutated, replaced, or deleted.

Evidence (immutable evidence). *Evidence* denotes immutable artefacts referenced by the registry. Within OPS-1, evidence is context-free: the Engine Core does not interpret evidence or derive semantics from it (see `I_evidence_immutable`).

Evidence is immutable after creation. It must not be overwritten, replaced, or deleted. Any content change constitutes a different artefact and is admissible only as a new, separately referencable evidence artefact.

4.3 State and transitions

The Engine Core is a stateless evaluator with a terminal boundary. For a single case, the kernel admits exactly one abstract step:

$$(S, \mathcal{I}, \Theta) \Rightarrow R$$

There is no internal iteration, retry policy, recovery mechanism, or adaptive transition structure (see `I_nonadaptive`, `P_feedback`, `P_learning`).

No hidden state rule (invariant). The kernel must not depend on hidden or mutable state between runs (e.g. caches, learned parameters, implicit memory). Any such dependency is non-conformant with OPS-1 (see `I_nonadaptive`, `P_learning`, `D_adaptive_ext`).

4.4 Threshold objects (Θ) and regions

Thresholds Θ define admissible regions **only through** invariant predicates. The Engine Core does not interpret thresholds as targets, goals, or optimization criteria (see `P_optimization`).

Region notion (structural). A *region* is the induced set of snapshots satisfying a given invariant set:

$$\Omega(\mathcal{I}) := \{S \mid \forall I_k \in \mathcal{I} : I_k(S) = 1\}$$

Regions encode structural admissibility only. They are not search spaces, performance landscapes, or optimization domains (see `P_optimization`).

4.5 STOP as first-class outcome

STOP is a first-class terminal outcome:

$$\text{STOP} = 1 \Rightarrow \text{no further diagnostic evaluation is admissible.}$$

STOP is not an error surface. STOP is not a recoverable failure, not a warning, and not advice. It denotes a formal boundary beyond which diagnostic evaluation is invalid (see Section 7).

Classification categories (minimal).

- `admissible`,
- `boundary-violating`,
- `non-diagnosable`,
- `stop-eligible`.

Category labels are structural identifiers only. They must not be ranked, mapped to prescriptions, or treated as decision outputs (see `I_diagonly`, `P_policy`).

5 Interfaces and I/O Contracts

This section defines the **structural** input/output contracts of the Engine Core as a **diagnostic-only** kernel. The contracts are minimal and closed. They permit reproducible diagnostic evaluation and **explicitly prohibit** control, optimization, adaptation, learning, ranking, prioritization, guidance, and any prescriptive or normative semantics.

5.1 Inputs (schemas, required fields)

The Engine Core consumes exactly one evaluation request:

$$Q := \langle S, \mathcal{I}, \Theta, M \rangle$$

where S is the state snapshot, $\mathcal{I}$ the declared invariant set, Θ the declared threshold objects, and M a minimal metadata envelope.

Required input components (invariant).

- **State snapshot S .** Immutable and read-only. Partiality is admissible **only if** missing fields are explicitly represented in the observability descriptor $\mathcal{O}$.
- **Invariant set $\mathcal{I}$.** Explicit, versioned, and inspectable predicates $I_k : S \rightarrow \{0, 1\}$.
- **Threshold objects Θ .** Explicit, typed boundary parameters referenced by invariants. Thresholds are evaluation inputs only. They are **never** tuned, adapted, calibrated, optimized, inferred, or learned by the kernel.
- **Metadata envelope M .** Minimal contextual metadata such as `case_id`, `engine_version`, and stable identifiers of the input set. Metadata **must not** influence evaluation, classification, STOP semantics, admissibility, or provenance identity.

Observability as explicit input (prohibition). Unknown or indeterminate values **must** be represented explicitly (e.g. via $\mathcal{O}$). The kernel **must not** replace missing observability by assumptions, defaults, heuristics, interpolation, estimation, probabilistic completion, or implicit normalization.

5.2 Outputs (artefacts, schemas)

The Engine Core produces exactly one evaluation result:

$$R := \langle \mathbf{v}, \mathbf{c}, \text{STOP}, P \rangle$$

with:

- $\mathbf{v} \in \{0, 1\}^n$ — invariant satisfaction vector;
- $\mathbf{c}$ — finite structural classification label;
- $\text{STOP} \in \{0, 1\}$ — terminal STOP flag;
- P — minimal **append-only** provenance record.

No prescriptive or normative fields (prohibition). Outputs **must not** contain recommendations, priorities, rankings, weights, scores, objectives, targets, success criteria, confidence levels, action proposals, or decision guidance. The presence of any such field, annotation, default ordering, or implicit weighting **invalidates** OPS-1 conformance.

Provenance payload (minimal). At minimum, the provenance record binds outputs to inputs via stable identifiers:

$$P := \langle h(S), h(\mathcal{I}), h(\Theta), \mathbf{v}, \mathbf{c}, \text{STOP} \rangle$$

External systems may attach additional identifiers (e.g. run IDs or build hashes) **only if** they introduce no new semantics and do not alter classification, STOP meaning, admissibility boundaries, ordering, weighting, or provenance identity.

5.3 Error surfaces (typed failure outputs)

The Engine Core strictly distinguishes **terminal STOP** from **tooling or implementation errors**.

STOP (kernel-level invariant). STOP is a first-class, terminal boundary of diagnostic admissibility (see Section 7). It is **not** a recoverable error, **not** a warning, and **not** advice. It **does not** signal optimization, remediation, or corrective action.

Implementation errors (non-kernel). Failures such as parsing errors, missing files, corrupted input containers, or environment misconfiguration are *external* to Engine Core semantics. They may be represented by surrounding tooling as typed errors, but:

- they **must not** be conflated with STOP;
- they **must not** introduce recovery, retry, fallback, or heuristic logic into the kernel;
- they **must not** affect invariant evaluation, classification, STOP semantics, or admissibility boundaries.

The kernel defines R **only** for admissible inputs. Input admission, error handling, and tooling reliability are external concerns.

5.4 Extension points and compatibility boundary (OPS-1)

Allowed extension points (closed list). The following extension points are **explicitly allowed**. This list is **closed**. Any extension not listed here is **non-admissible** under OPS-1.

- **Presentation or rendering layers** (strictly cosmetic; no semantic, normative, or decision effect).
- **Non-normative reporting views** (read-only display of existing artefacts; no interpretation).
- **Evidence import adapters** (syntactic only; no semantic transformation; full provenance).
- **Additional verification gates** (binary only; stricter is admissible; weaker or graded is prohibited).
- **Witness packaging** (bundling artefacts only; no selection, reordering, or narrative).

Semantic neutrality constraint (invariant). Every allowed extension is admissible **only if** it introduces **no new semantic degrees of freedom**. In particular, an extension **must not**:

- imply preference, recommendation, priority, or importance;
- introduce ranking, scoring, weighting, color semantics, or “default views” that act as a normative lens;

- alter classification, STOP semantics, provenance identity, ordering, or admissibility boundaries.

Binary conformance rule (invariant). Conformance is **binary**: conformant or *non-ESTRA*. Any layer that can change a reader’s decision preference without changing the underlying computed artefacts is **non-ESTRA**, regardless of tooling quality, intent, or presentation polish.

Toolkit / CLI / GUI boundary (OPS-2 as consumer only). Any toolkit surface (CLI, GUI, scripts, integrations) is outside OPS-1 and belongs to OPS-2. OPS-2 is an **exchangeable consumer** of the OPS-1 contract. It may transport or render Q and R , but it **must not** define, reinterpret, aggregate, rank, operationalize, or extend OPS-1 semantics.

OPS-2 variation is admissible **only** within the closed extension space above. No interface layer may weaken invariants or gates, introduce control semantics, feedback loops, learning, heuristic shortcuts, ranking, scoring, or STOP bypass.

Auto-disqualification by semantic contamination. If any layer (including documentation, examples, defaults, or positioning) represents the Engine Core as a “tool”, “method”, “framework”, “best practice”, “playbook”, or “how-to”, that representation is **non-conformant** by definition. If any layer induces or implies “you should do X” from R or from STOP, it introduces prescriptive semantics and is therefore **non-ESTRA**.

5.5 Versioning and compatibility contract

Version identifiers (invariant). The Engine Core **must** expose a version identifier (e.g. content hash or semantic version) via metadata M and/or provenance P .

Compatibility rule (minimal, invariant). Results are comparable as kernel outputs **only if**:

- $h(\mathcal{T})$ is identical;
- $h(\Theta)$ is identical;
- the Engine Core version identifier is identical.

If any of these differ, comparison is external to OPS-1 and is **not** a kernel property.

No implicit upgrades (prohibition). The kernel **must not** silently reinterpret invariants, thresholds, classifications, ordering, or STOP semantics across versions. Any change **requires** an explicit version change and updated identifiers.

6 Execution Model and Determinism

This section specifies the execution model of the Engine Core strictly as a **diagnostic-only** evaluator. It **excludes** any notion of process orchestration, workflow control, scheduling, runtime management, or operational governance.

6.1 Run lifecycle

A single kernel run is defined as one atomic evaluation:

$$(S, \mathcal{I}, \Theta, M) \Rightarrow R$$

where M denotes explicit execution metadata (e.g. version identifiers or environment descriptors). Such metadata is recorded for traceability **only** and **must not** influence evaluation, classification, STOP semantics, or provenance identity.

There is exactly one input bundle and exactly one result. The Engine Core **does not** manage sessions, pipelines, retries, recovery, or long-lived execution contexts.

No internal lifecycle (invariant). The kernel has **no** notion of *start*, *progress*, or *completion* beyond the existence of the evaluation result R or a terminal STOP. Any higher-level lifecycle management is external and **non-normative** with respect to OPS-1.

6.2 Determinism as architectural invariant

OPS-1 fixes determinism as an **architectural invariant** of the kernel. Determinism is **not** a best-effort execution property, **not** a process guarantee, and **not** a toolchain attribute. It is a necessary and non-negotiable condition for OPS-1 conformance.

Deterministic domains (invariant). For identical declared inputs $(S, \mathcal{I}, \Theta, M)$, the kernel **must** yield an identical result R in all of the following domains:

- **Diagnostic result determinism:** identical admissibility classification and identical diagnostic outputs.
- **Operation graph traversal determinism:** traversal and evaluation order are determined solely by declared structures.
- **STOP detection determinism:** terminal STOP recognition is deterministic and input-complete.
- **Provenance assignment determinism:** attribution of outputs to declared inputs and referenced artefacts is deterministic.
- **Registry state determinism (logical):** registry evolution is append-only and order-stable in the logical sense of reference introduction.

Any deviation in any of these domains **violates** OPS-1 determinism.

What determinism does *not* mean (descriptive). Kernel determinism **does not** imply:

- predictability of future states of the diagnosed system;
- stability of the system over time;
- repeatability of real-world observations;
- causal explanation, attribution, or forecasting of outcomes.

Determinism is solely a property of admissibility evaluation within the kernel boundary.

6.3 Bounded non-determinism (strictly cosmetic)

OPS-1 permits non-determinism **only** where it has **no semantic impact** on diagnostic results, terminal STOP evaluation, provenance identity, or registry meaning.

Permitted (strictly bounded). The following are permitted **only** if they are demonstrably semantics-free:

- ordering of physically independent artefacts, **if and only if** ordering has no effect on diagnostic meaning;
- rendering and layout properties (purely cosmetic presentation);
- non-normative metadata that does not influence evaluation semantics.

Forbidden (disqualifying). Any non-determinism is **forbidden** if it can influence:

- diagnostic outputs or admissibility classification;
- terminal STOP recognition or NO-GO evaluation;
- provenance assignment or traceability links;
- selection of execution paths by heuristic, adaptive, or conditional logic;
- time-, load-, or environment-dependent behavior with semantic impact.

If such non-determinism is present or required, OPS-1 conformance is **violated**.

6.4 No hidden state rule

The Engine Core **must not** depend on hidden or mutable internal state.

Explicitly forbidden (prohibition).

- caches that affect evaluation results;
- learned parameters or adaptive thresholds;
- implicit memory across runs;
- time-dependent behavior not represented in inputs.

Any dependency not explicitly represented in $(S, \mathcal{I}, \Theta, M)$ **invalidates** OPS-1 conformance.

6.5 Environment independence (semantic)

The kernel is environment-independent in a *semantic* sense:

- evaluation semantics **must not** depend on wall-clock time;
- evaluation semantics **must not** depend on execution order of prior runs;
- evaluation semantics **must not** depend on external services or side effects.

This constraint applies exclusively to kernel semantics. It imposes **no** requirements on build tooling, containers, deployment environments, or infrastructure, all of which are external to OPS-1.

6.6 No execution control semantics

The Engine Core admits **no** execution control semantics of any kind. In particular, it does **not** admit:

- pause, resume, or step-wise execution;
- conditional branching or scenario switching;
- fallback, degraded, recovery, or continuation modes.

Execution yields exactly one admissible outcome: either a result R or a terminal STOP. No admissible third state exists.

7 Failure Modes and STOP Conditions

This section defines **terminal STOP** as a structural and irreversible boundary of diagnostic admissibility. STOP is **not** an error, **not** a warning, and **not** a recommendation. It denotes the condition under which further diagnostic evaluation is formally non-admissible.

7.1 Structural failures

A *structural failure* occurs when required preconditions for diagnostic evaluation are violated in a manner that invalidates admissibility itself.

Examples include:

- internal inconsistency of the invariant set $\mathcal{I}$;
- contradictions between declared invariants and threshold objects Θ ;
- violation of kernel invariants (see `I_nonintervention`, `I_nonadaptive`).

Structural failures are properties of the declared input configuration. They **trigger** terminal STOP immediately. They **do not** represent execution faults, runtime errors, or implementation failures.

7.2 Observability loss

Observability loss occurs when the state snapshot S lacks sufficient declared information to evaluate required invariants and this insufficiency is explicitly encoded in the observability descriptor $\mathcal{O}$.

No implicit completion. The kernel must not compensate observability loss by assumptions, defaults, heuristics, estimation, or probabilistic inference. This is a direct consequence of non-adaptivity and the prohibition of learning/inference semantics (see `I_nonadaptive`, `P_learning`).

If required observability is missing, diagnostic admissibility is violated and terminal STOP applies. This condition is *distinct* from provenance loss: observability concerns the declared completeness of S at evaluation time.

7.3 Threshold violations

Threshold violations occur when invariant predicates that reference declared threshold objects Θ evaluate to false.

Hard boundary semantics. Thresholds are treated as hard structural boundaries. There is no notion of soft constraints, grace margins, tolerances, or gradual degradation inside the kernel (see `P_optimization`). Any violation constitutes loss of admissibility and triggers terminal STOP.

7.4 STOP region semantics

STOP defines a terminal admissibility region in the diagnostic domain:

$$\text{STOP} = 1 \Rightarrow \nexists R' \text{ such that } R \prec R'$$

That is, no admissible diagnostic result exists that can succeed R for the same declared case.

Irreversibility. Irreversibility is structural, not temporal. STOP does not denote a moment in time, a process stage, or an execution phase. It denotes the absence of any admissible continuation within the kernel's domain (see `I_stop_mono`).

Monotonicity. Once STOP holds for a declared case, it remains true for that case. STOP cannot be downgraded, overridden, weakened, or reinterpreted without changing the declared inputs (see `I_stop_mono`).

Singularity. STOP is singular and absolute per case. There is no notion of partial, layered, probabilistic, conditional, or recoverable STOP (see `I_binary`).

7.5 Diagnostic operation graph (normative)

The Engine Core admits a single, strictly directed diagnostic operation graph. The graph is **acyclic**, **read-only**, and **terminal**. This graph is normative and closed.

Nodes. The diagnostic graph consists of the following abstract operation nodes:

- **InputDeclaration:** reception of $(S, \mathcal{I}, \Theta)$;
- **StructuralValidation:** validation of kernel invariants and input consistency;
- **InvariantEvaluation:** evaluation of declared invariants;
- **STOP:** terminal non-admissibility state;
- **ResultEmission:** emission of R with $\text{STOP} = 0$.

Edges (allowed). The only admissible directed edges are:

$$\text{InputDeclaration} \rightarrow \text{StructuralValidation} \rightarrow \text{InvariantEvaluation} \rightarrow \{\text{STOP}, \text{ResultEmission}\}$$

STOP as terminal sink. STOP is a terminal sink node. There exists no outgoing edge from STOP (see `I_stop_mono`).

Forbidden edges. The following edges are explicitly forbidden:

- any edge leaving **STOP**;
- any feedback, backward, or cyclic edge between any nodes (see **P_feedback**, **I_nonadaptive**);
- any retry, recovery, continuation, or re-evaluation edge;
- any edge conditioned on optimization, preference, ranking, or goal criteria (see **P_optimization**, **P_policy**);
- any edge introducing control, steering, learning, or adaptive behavior (see **P_control**, **P_learning**).

Any artefact implying a diagnostic graph outside this definition is non-admissible under OPS-1.

7.6 Micro-instantiation (formal traces)

The following minimal traces illustrate how violations of OPS-1 constraints necessarily trigger terminal **STOP**. They are semantic counterexamples, not implementation examples.

Trace A: Determinism violation. Given identical declared inputs $(S, \mathcal{I}, \Theta)$, assume evaluation produces two distinct results $R \neq R'$. This violates determinism (**I_det**) and immediately triggers auto-disqualification (**D_nondet**), hence **STOP**.

Trace B: Provenance loss. Assume an evaluation result lacks a complete append-only provenance record. This violates provenance preservation (**I_prov**) and triggers auto-disqualification (**D_prov_loss**), hence **STOP**.

Trace C: Forbidden extension. Assume an extension introduces behavior dependent on past evaluations. This introduces feedback or adaptation (**P_feedback**, **P_learning**), violating non-adaptivity (**I_nonadaptive**) and triggering auto-disqualification (**D_adaptive_ext**), hence **STOP**.

7.7 Minimal audit trail for **STOP**

When terminal **STOP** is triggered, the kernel records a minimal audit trail in the append-only provenance record P :

- identifier of the input snapshot $h(S)$;
- identifier of the invariant set $h(\mathcal{I})$;
- identifier of the threshold objects $h(\Theta)$;
- the structural classification label $\mathbf{c}$;
- the **STOP** flag.

This audit trail exists solely to support traceability and conformance checks. It must not be extended to provide causal narratives, remediation advice, recovery logic, decision guidance, or policy justification.

8 Validation and Warning Gates

This section defines the **conformance gates** that determine whether a PDF-4 Engine Core artefact is admissible as OPS-1 compliant. These gates apply to the *artefact and its build process*, not to the diagnosed system and not to any external tooling that consumes diagnostic outputs.

8.1 Build gate (EXIT_CODE)

The primary conformance gate is the build gate.

Rule. An OPS-1 admissible artefact must build successfully with:

$$\text{EXIT_CODE} = 0$$

Any non-zero exit code renders the artefact non-admissible, independent of its intended semantics or declared scope.

Rationale. A failing build indicates structural inconsistency or ambiguity in the artefact definition and therefore invalidates it as a normative specification.

8.2 True warnings gate (strict)

Definition. *True warnings* are warnings emitted by the document toolchain (L^AT_EX, packages, classes) that indicate semantic ambiguity, structural inconsistency, or unresolved references.

Rule. The following warning classes must be empty:

- LaTeX Warning;
- Package Warning;
- Class Warning.

If any true warning is present, the artefact fails conformance validation.

Scope. This gate concerns only ambiguity in the specification artefact itself. It does not assert correctness of external implementations or interpretations.

8.3 Allowed cosmetic noise (non-gated)

The following diagnostics are explicitly *non-gated*:

- overfull or underfull box warnings;
- line-breaking or spacing diagnostics;
- font substitution notices with identical metrics.

These messages may be recorded for editorial purposes but must not affect OPS-1 conformance status.

8.4 Artefact completeness checks

An OPS-1 admissible PDF-4 artefact must satisfy the following structural completeness conditions:

- all referenced sections exist and are non-empty;
- all cross-references resolve without warnings;
- no placeholder markers (e.g. `TODO`) remain in normative sections;
- section ordering matches the declared document structure.

Completeness checks assess *structural integrity only*. They do not evaluate semantic adequacy, empirical validity, or interpretive quality.

8.5 Operation graph conformance gate

Rule. The specification must not imply any diagnostic operation graph other than the strictly directed, acyclic graph defined in Section 7.

Failure conditions. The artefact is non-admissible under OPS-1 if it implies:

- cyclic diagnostic paths;
- feedback, control, retry, or recovery edges;
- non-terminal or reversible STOP semantics;
- adaptive, heuristic, or decision-based branching.

Scope. This gate is structural and declarative. It is independent of any concrete implementation strategy and concerns admissibility of the specification only.

9 Reproducibility and Traceability

This section specifies the minimal reproducibility and traceability guarantees required of an OPS-1 Engine Core artefact. These guarantees are **structural**. They concern the ability to reproduce diagnostic results from declared inputs only. They **do not** concern the empirical behavior of the diagnosed system.

9.1 Reproducibility as architectural invariant (binary)

OPS-1 treats reproducibility as a **binary architectural property**: an artefact is either reproducible or non-reproducible. No intermediate status, gradation, or confidence level is admissible.

Necessary condition for validity (invariant). Given identical declared inputs, the Engine Core **must** reproduce identical diagnostic results. If this condition fails, the artefact is **invalid** under OPS-1.

Not a process property (descriptive). Reproducibility is **not** a matter of process discipline, execution hygiene, toolchain choice, or environment management. It is fixed exclusively at the level of kernel semantics and declared inputs.

Non-reproducibility implies terminal STOP-admissibility (invariant). If reproducibility cannot be established for an output, diagnostic admissibility is **violated**. The situation is terminal STOP-admissible, and the output **must not** be treated as an OPS-1 result.

9.2 Registry and evidence as architecture (non-technical)

OPS-1 treats the **append-only registry** and **immutable evidence** as architectural constructs, not as tooling, storage systems, or processes.

Registry (logical, reference-only). The registry is a logical reference space that contains *references* and *links* only. It carries **no** interpretation, **no** aggregation semantics, and **no** decision or filter logic. Registry contents **must not** influence evaluation, classification, terminal STOP semantics, or admissibility boundaries.

Evidence (immutable, context-free). Evidence denotes immutable artefacts referenced by the registry. Evidence is **context-free** in OPS-1 kernel semantics. Within OPS-1, the Engine Core **must not** interpret, normalize, clean, sample, filter, aggregate, or otherwise transform evidence.

Evidence is immutable after creation. It **must not** be overwritten, replaced, or deleted. Any content change **constitutes** a different artefact and is admissible **only** as a new, separately referencable evidence artefact.

9.3 Append-only invariants

Append-only registry rule (invariant). Registry entries are append-only. Once a reference or link exists, it **must not** be mutated, replaced, reordered, or deleted. Any registry that permits mutation is **non-admissible** under OPS-1.

Order-stable registry semantics (invariant). Registry introduction order is semantically stable in the logical sense that references, once introduced, remain introduced. No re-ordering is permitted to change meaning, provenance identity, or diagnostic interpretation.

Append-only evidence referencing (invariant). Once an evidence artefact is referenced, that reference **must** remain valid without post-hoc alteration. Any later artefact with changed content **must** be treated as a distinct evidence artefact and referenced separately.

9.4 Provenance (complete or invalid)

Each diagnostic result R **must** be traceable to its declared inputs via a **complete** provenance mapping:

$$(S, \mathcal{I}, \Theta) \mapsto R$$

Completeness requirement (invariant). Provenance is **complete or invalid**. There is **no** admissible notion of partial, probabilistic, approximate, inferred, or reconstructed provenance within OPS-1. If any required link is missing, provenance is **invalid**.

Transitive traceability (invariant). Every diagnostic output **must** be transitively traceable to:

- the specific input snapshot S ;
- the specific invariant set $\mathcal{I}$;
- the specific threshold objects Θ ;
- the Engine Core version identifier;
- all evidence artefacts referenced for that output (if any).

This traceability requirement concerns *referencing* and *link integrity* only. It provides **no** explanatory narrative, causal account, or justification.

9.5 Stable identifiers and manifests (format-agnostic)

The Engine Core **must** expose stable identifiers for:

- input state snapshots;
- invariant sets;
- threshold objects;
- the Engine Core version.

The choice of identifier algorithms, encodings, manifests, or serialization formats is external to kernel semantics, provided identifiers are stable, inspectable, and unambiguous.

9.6 Explicit prohibitions (non-negotiable)

The following are **explicitly non-admissible** under OPS-1:

- **Mutable registry:** any ability to edit, rewrite, replace, reorder, or delete registry entries.
- **Evidence normalization or cleaning:** any transformation of evidence content that changes meaning, regardless of labeling.
- **Aggregation with semantic effect:** any aggregation that alters the meaning of evidence or changes diagnostic interpretation.
- **Post-hoc correction of evidence:** any after-the-fact repair, replacement, rewriting, or retrofitting of evidence to enforce consistency.
- **Opaque preprocessing:** any uninspectable transformation or hidden intermediate that breaks traceability.
- **Hidden sampling or filtering:** any undisclosed selection, exclusion, prioritization, or weighting that prevents complete provenance.
- **Semantically relevant non-determinism:** any non-determinism that can influence diagnosis, terminal STOP evaluation, or provenance identity.

These prohibitions are **architectural invariants**. They are independent of implementation strategy and **must** hold for OPS-1 conformance.

10 Limits and Non-Goals

This section defines the **explicit limits** of the Engine Core and fixes what is intentionally excluded from PDF-4. These limits are **constitutive**. They define the boundary of admissible interpretation for OPS-1 diagnostic semantics.

10.1 Interpretation boundaries

Diagnostic outputs produced by the Engine Core are **structural classifications only**. They express admissibility or non-admissibility with respect to declared invariants and terminal STOP conditions.

Any interpretation beyond invariant satisfaction is **external** to the kernel and has **no standing** within OPS-1.

No semantic lifting (prohibition). Consumers of diagnostic output **must not** treat classifications or STOP as:

- recommendations, guidance, or action directives;
- root-cause analyses or explanatory narratives;
- forecasts, predictions, or risk estimates;
- compliance judgments, managerial assessments, or performance evaluations;
- priorities, rankings, weights, targets, or any notion of “what matters most”.

Any such interpretation requires additional assumptions, evidence, or authority and is **excluded** from the Engine Core boundary.

10.2 OPS-1 vs. OPS-2 responsibility split

OPS-1 is a **contract and authority layer**. It fixes admissibility, invariants, terminal STOP semantics, determinism, reproducibility, and provenance identity.

OPS-1 **does not** address usage, presentation, tooling, orchestration, or decision-making.

Any toolkit surface (CLI, GUI, scripts, integrations) belongs to OPS-2 and is an **exchangeable consumer** of OPS-1. OPS-2 may display, transport, or package results, but it **must not** define, reinterpret, operationalize, aggregate, rank, or extend OPS-1 semantics.

Binary compliance (invariant). Compliance is **binary**: conformant or *non-ESTRA*. Intent, usability goals, product considerations, or domain expectations **do not** soften this boundary.

Auto-disqualification by category error. Any representation of OPS-1 as a “method”, “tool”, “framework”, “best practice”, “playbook”, or “how-to” is a category error. If any OPS-2 layer (or its documentation) implies “you should do X” from a classification or from terminal STOP, it introduces prescriptive semantics and is therefore **non-ESTRA**.

10.3 Known non-diagnostic cases

The Engine Core **cannot** produce admissible diagnostic results when:

- required observability is missing, indeterminate, or explicitly absent;
- invariants are inconsistent, underspecified, or contradictory;
- threshold objects are ambiguous, incompatible, or ill-defined;
- kernel invariants are violated;
- provenance completeness cannot be established;
- determinism or reproducibility, as architectural invariants, is violated.

In all such cases, diagnostic admissibility itself is **violated** and terminal STOP **applies**. STOP does **not** compensate for these conditions; it denotes their formal recognition.

10.4 What requires external evidence or authority

The following concerns **necessarily require** evidence, processes, or authority **outside** the Engine Core:

- remediation planning or corrective action design;
- prioritization, sequencing, or trade-off resolution;
- assignment of organizational responsibility or accountability;
- economic, performance, or impact evaluation;
- governance, approval, or decision authority.

PDF-4 **explicitly declines** to address these concerns in order to preserve diagnostic neutrality and ontological integrity of the kernel.

10.5 Explicit exclusion of workflows and methods

PDF-4 **does not** define workflows, procedures, methods, playbooks, recipes, or execution instructions of any kind.

The diagnostic operation graph defined in PDF-4 exists **solely** to fix admissibility boundaries and terminal STOP enforcement. It **must not** be interpreted as:

- a process model or workflow description;
- a decision tree or control structure;
- an execution sequence to be followed by agents or systems;
- a method for optimization, steering, intervention, or management.

Any interpretation of the operation graph as a workflow, method, or procedural guidance is **non-admissible** under OPS-1 and outside the declared scope of PDF-4.

10.6 Determinism and reproducibility are not process-negotiable

OPS-1 fixes determinism and reproducibility as **architectural invariants**. Accordingly, PDF-4 **does not** define, recommend, imply, or tolerate any process-level mechanisms intended to “achieve”, “improve”, or “enforce” them.

If determinism or reproducibility would require procedural controls to hold, the artefact is **non-admissible** under OPS-1 and produced outputs are **invalid**.

10.7 Forbidden extensions (auto-disqualifying)

The following extension classes are **explicitly forbidden**. If any is present, the artefact is **NOT ESTRA-conformant**.

Semantic control and steering.

- any optimization, recommendation, prioritization, ranking, weighting, scoring, targeting, or decision semantics;
- any control logic influencing or steering diagnostic outcomes;
- any feedback edge or adaptive branching affecting evaluation.

Learning and adaptation.

- any learning, fitting, calibration, adaptation, or tuning influencing invariants, thresholds, classification, provenance identity, admissibility, or terminal STOP;
- any use of historical cases to modify kernel behavior.

STOP violation.

- any retry, recovery, bypass, continuation path, or “soft STOP” after terminal STOP;
- any reinterpretation of STOP as warning, error, signal, advisory, or “needs review”.

Non-determinism and provenance violation.

- any semantically relevant non-determinism;
- any partial, probabilistic, inferred, reconstructed, or narrative provenance;
- any post-hoc correction, repair, normalization, or retrofitting of evidence or diagnostic artefacts.

Registry and evidence manipulation.

- mutable registries or evidence stores;
- overwriting, replacing, deleting, rewriting, or reordering evidence based on relevance or “importance”;
- opaque preprocessing, hidden sampling, hidden filtering, or undisclosed selection criteria.

Renderer and reporting semantics (presentation-driven meaning).

- any visualization, UI, report, or default view that implies preference, importance, priority, “severity”, “criticality”, or “health” from OPS-1 outputs;
- any ranking, top- N , ordering by “risk”, “impact”, or “value”, heatmap semantics, traffic-light semantics, or highlight rules acting as a normative lens;

- any report text interpreting outputs as guidance, recommended action, or procedural next step.

Evidence import semantics (adapter-driven meaning).

- any import adapter that filters, cleans, normalizes, imputes, smooths, deduplicates by relevance, treats outliers, or otherwise transforms evidence in a semantics-bearing way;
- any adapter behavior where independent implementers ingest the same external evidence and obtain materially different semantics without an explicitly declared, reversible mapping rule.

Graded or stratified verification (gate-driven meaning).

- any “quality levels”, “maturity tiers”, or graded acceptance (e.g. pass-with-advice, bronze/silver/gold), regardless of naming;
- any gate output acting as a score, ranking, or preference signal rather than a strict binary predicate (pass/fail).

Witness packaging semantics (packaging-driven narrative).

- curated subsets, “highlights”, “executive packs”, “most important artefacts”, or any selection or reordering intended to create narrative, weight, or reading guidance;
- any packaging that changes meaning by emphasizing, suppressing, or sequencing artefacts.

Binary auto-disqualification rule (invariant). If any extension can change a reader’s decision preference without changing the underlying computed artefacts, it introduces normative semantics and is therefore **non-ESTRA**.

Presence of any forbidden class **immediately disqualifies** the artefact from OPS-1 and ESTRA conformance, regardless of tooling, intent, or external justification.

10.8 Contextual positioning and non-normative references

The following references are provided solely to situate OPS-1 within a broader landscape of formal and diagnostic systems. They are cited for context only. They are **not** adopted, **not** inherited, and **not** normatively binding.

- Formal specification and verification: Alloy, TLA⁺, Coq — predicate-based invariants and non-constructive refutation.
- Provenance and traceability: W3C PROV, append-only ledger models — structural lineage without causal interpretation.
- Enterprise architecture descriptions: ISO/IEC 42010, NIST architecture frameworks — contrasted by OPS-1’s non-prescriptive scope.
- Epistemic boundaries: Popperian falsifiability and strict refutation as opposed to optimization or solution synthesis.

OPS-1 intentionally diverges from these traditions by enforcing a strictly diagnostic, non-interventionist, binary admissibility kernel. No comparative performance, coverage, or superiority claim is made.

11 Appendix: Manifest

This appendix records the structural manifests required to assess PDF-4 as a review-ready OPS-1 artefact. The manifests are **descriptive** and **non-executable**. They introduce **no** normative, prescriptive, or methodological semantics.

11.1 File manifest

The PDF-4 artefact consists of the following files and sections:

- `main.tex` — document entry point and section ordering;
- `sections/00_abstract.tex` — positioning and scope summary;
- `sections/01_problem_statement.tex` — motivation and exclusions;
- `sections/02_engine_scope_and_nonclaims.tex` — kernel boundary;
- `sections/03_core_objects_and_invariants.tex` — objects and invariants;
- `sections/04_engine_interfaces_and_io_contracts.tex` — I/O contracts;
- `sections/05_execution_model_and_determinism.tex` — execution semantics;
- `sections/06_failure_modes_and_stop_conditions.tex` — STOP semantics;
- `sections/07_validation_and_warning_gates.tex` — conformance gates;
- `sections/08_reproducibility_and_traceability.tex` — provenance rules;
- `sections/09_limits_and_non_goals.tex` — explicit limits.

No additional normative content exists outside this list.

OPS-2 is external by definition (invariant). Any toolkit surface (CLI, GUI, scripts, integrations), documentation, product framing, or usage guidance belongs to OPS-2 and is **not** part of this OPS-1 artefact. Such surfaces may consume the OPS-1 contract but **must not** be treated as sources of OPS-1 authority or semantics.

11.2 Build manifest

The artefact is considered valid **only if**:

- the document builds with `EXIT_CODE = 0`;
- no true warnings (LaTeX/Package/Class) are emitted;
- all cross-references resolve;
- no placeholder markers remain in normative sections.

Build tooling, environments, and containerization are external to kernel semantics and are therefore **not** specified here.

11.3 Schema manifest

The Engine Core relies on the following conceptual schemas:

- *State snapshot schema* for S (fields, values, observability);
- *Invariant schema* for $\mathcal{I}$ (predicates over S);
- *Threshold schema* for Θ (typed boundary parameters);
- *Result schema* for R (classification and STOP flag);
- *Provenance schema* for P (append-only identifiers).

Concrete serialization formats (YAML, JSON, tables, etc.) are intentionally unspecified. Any format is admissible **only if** it preserves structure, identity, and traceability.

No method or tool semantics by serialization (prohibition). No serialization format, wrapper, or consumer layer **may** add prescriptive fields, recommendations, or “how-to” semantics to the OPS-1 contract. Any such addition is external and **non-conformant** to OPS-1 by definition.